

THE PRODUCTION AI PLAYBOOK

5 Reasons AI Features Fail in Production

...and how to fix them

95% of enterprise GenAI pilots never deliver measurable impact — and the reasons are surprisingly predictable. This is the short, practical guide to the five that sink most teams, with a concrete fix for each.



Yovany Luis

AI Product Engineer · 8+ yrs · ex-Immunefi, Thrive Market

yovany.dev

WHY THIS MATTERS

Most AI features **never make it** to a result that matters.

95%

of enterprise GenAI pilots fail to deliver measurable business impact.

MIT · STATE OF AI IN BUSINESS
2025

80%

of AI projects miss their intended business goal — twice the rate of non-AI software.

RAND CORPORATION · 2025

77%

of failures are **not** the model — they're strategy, integration & operations.

FOLIO3 · 140 DEPLOYMENTS

WHERE THE 80% ACTUALLY GOES (RAND, 2025)

Outcome of AI projects

share of all projects

33.8%

28.4%

18.1%

19.7%

■ Abandoned before production ■ Shipped, no business value ■ Some value, cost not justified ■ Met or beat the goal

→ The good news: the failure modes are **predictable and fixable**. Only ~1 in 5 projects clears the bar today — but the gap is rarely raw model quality. It's the engineering and product discipline around the model. The next five pages are the five places teams slip, and exactly how to recover each one.

01	You ship on vibes, not evals	p.3
02	It sounds confident — but it's wrong	p.4
03	You're flying blind in production	p.5
04	Cost & latency spiral out of control	p.6
05	You built the wrong thing	p.7

01

QUALITY & TESTING

You ship on vibes, not evals

THE SYMPTOM

"It looked great in the demo." Then real users hit edge cases and quality quietly slides — and you can't tell whether your last prompt tweak helped or hurt. **73% of failed AI projects never defined what "good" meant before launch.**

WHY IT HAPPENS

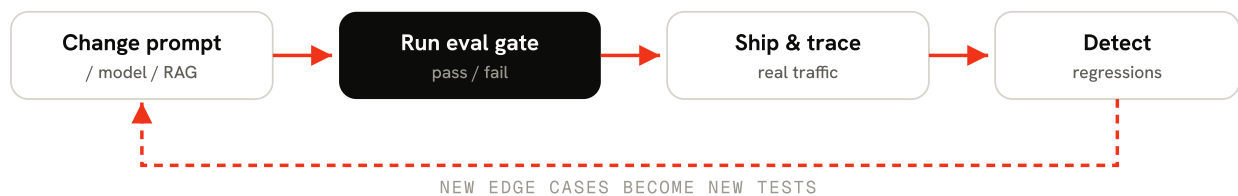
LLM output is non-deterministic. Without a test set, every prompt or model change is a guess and every regression is invisible until a user finds it. Manual spot-checking doesn't scale past a handful of cases.

THE FIX — BUILD AN EVAL LOOP

- ✓ Write **20–50 real example inputs** with expected behavior *before* you ship.
- ✓ Score them automatically on every change — **deterministic checks + LLM-as-judge.**
- ✓ **Gate deploys on the suite.** No green, no ship.
- ✓ Feed real production traffic back into the test set as it grows.

THE EVALUATION LOOP

THE FASTER IT SPINS, THE BETTER THE PRODUCT GETS



★ **The one move:** in 2026, building production AI is mostly the discipline of running a tight eval-and-observability loop and picking sensible defaults for everything else. Start the loop before you write the feature — not after it breaks.

02 It sounds confident — but it's wrong

● THE SYMPTOM

The model invents facts, fields, or steps — *fluently*. Measured hallucination rates run **50–82%** depending on the model and task, and a confident wrong answer is the hardest failure to catch.

● WHY IT HAPPENS

Teams treat hallucination as one bug. It's **four**. Reaching for RAG on all of them adds latency and cost while fixing only the first. Match the fix to the failure type.

THE FIX

- ✓ **Ground** answers in retrieved, **cited** sources.
- ✓ **Verify entailment** — check the answer is actually supported (faithfulness score).
- ✓ **Constrain format** with schemas / structured output, not hope.
- ✓ Show sources so users can **catch** errors, not trust them.

HALLUCINATION ISN'T ONE PROBLEM — IT'S FOUR

Factual confabulation

Invents facts not in any source.

RAG + CITATIONS

Instruction drift

Ignores or bends what you asked.

PROMPT + EVALS

Format instability

Breaks the expected JSON / shape.

STRUCTURED OUTPUT

Context inconsistency

Contradicts earlier turns / data.

MEMORY + GUARDRAILS

! RAG is powerful — but it only addresses **factual confabulation**. Applying it to the other three burns latency and budget without reducing the real risk.

GROUNDING ANSWER PIPELINE

RETRIEVE → ANSWER → VERIFY BEFORE IT REACHES THE USER



03

OBSERVABILITY

You're flying blind in production

THE SYMPTOM

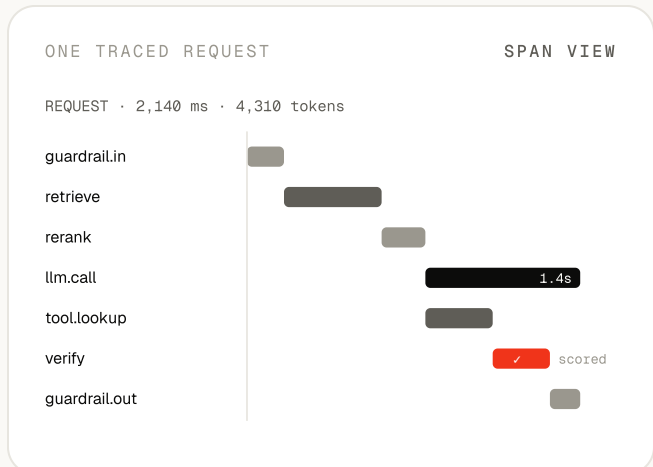
Your dashboards are all green — 200s, low latency, no errors — yet the feature is quietly giving users wrong, off-brand, or unsafe answers. You hear about it from a churned customer, not an alert.

WHY IT HAPPENS

Quality problems don't throw exceptions. Standard APM watches infra; it can't see that the answer was confidently wrong. Without tracing the actual prompts, tokens and outputs, the most important failure mode is invisible.

THE FIX — TRACE & SCORE LIVE TRAFFIC

- ✓ **Trace every request** — prompt, model, tokens, latency, tool calls, output (OpenTelemetry spans).
- ✓ **Score a sample of live traffic** on faithfulness, relevance & safety — not just dev sets.
- ✓ **Alert on drift & quality**, not only downtime.



→ Every span carries its prompt, tokens, latency and a quality score. Now "it feels worse this week" becomes a number you can **chart, alert on, and fix**.

THE THREE SIGNALS TO WATCH — TOGETHER

Quality**Cost / tokens****Latency**

Watch any one alone and you'll trade quality for speed, or speed for cost, without noticing. Observability means seeing all three on the same trace.

04

COST & LATENCY

Cost & latency spiral out of control

● THE SYMPTOM

The demo was cheap; production isn't. Infrastructure costs run **3–5× initial projections at scale**, token spend spikes unpredictably, and responses get slow enough that users abandon mid-task.

● WHY IT HAPPENS

Every request hits the biggest model, every time, with no caching, no routing and no budget. Token usage is dynamic and compounds with traffic — so cost and latency scale faster than usage.

THE FIX — A GATEWAY IN FRONT OF THE MODEL

- ✓ **Cache** aggressively (exact + semantic) for repeated queries.
- ✓ **Route by complexity** — small model for easy, large for hard.
- ✓ **Stream** responses so perceived latency drops.
- ✓ Add **fallbacks & per-feature budgets** at the gateway.

SPEND PER 1K REQUESTS

ILLUSTRATIVE

Every request → biggest model

100%

Cache + route + stream

~37%

↓ up to ~60% cost & latency cut

THE REQUEST FUNNEL

MOST TRAFFIC SHOULD NEVER REACH THE BIG MODEL

1,000 requests



EACH STAGE CATCHES WHAT THE LAST COULDN'T — THE COSTLY MODEL SEES THE FEWEST REQUESTS

05

PRODUCT & INTEGRATION

You built the wrong thing

● THE SYMPTOM

A technically working feature that nobody adopts. MIT calls it the **"learning gap"** — the model is fine, but it never fit a real workflow, so it stalls in pilot forever.

● WHY IT HAPPENS

Budgets chase flashy demos instead of boring, high-frequency work where the ROI actually is. And **61% of projects never measure ROI after launch** — so no one notices it isn't landing.

THE FIX — NARROW, EMBEDDED, MEASURED

- ✓ Pick **one narrow, high-frequency workflow** with a clear metric.
- ✓ Define **success up front** and measure ROI *after* launch.
- ✓ **Embed into existing tools**; keep a human in the loop where stakes are high.
- ✓ **Buy/partner** for commodity capability; build where you differentiate.

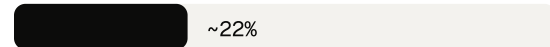
BUY VS. BUILD

SUCCESS RATE

Partner / buy from specialists



Internal build from scratch



SPEND VS. ROI

THE MISMATCH

Where most budget goes →

Sales & marketing demos

Where ROI is highest →

Back-office automation



The pattern behind all five: the model is rarely the bottleneck. Reliability, visibility, cost control and fit are. Treat your AI feature like a product with an SLA — not a demo — and you're already in the 5%.

THE ONE-PAGE CHECKLIST

Is your AI feature **actually** ready?

01 Evals before vibes

- ☐ 20–50 real test cases with expected behavior exist.
- ☐ They run automatically and gate every deploy.

02 Grounded & verified

- ☐ Answers cite retrieved sources; faithfulness is checked.
- ☐ Output format is enforced by a schema, not luck.

03 Observable in production

- ☐ Every request is traced (prompt, tokens, latency, output).
- ☐ Live traffic is scored; alerts fire on quality & drift.

04 Cost & latency under control

- ☐ Caching + model routing + streaming are in place.
- ☐ Per-feature budgets and fallbacks exist at a gateway.

05 Right problem, measured

- ☐ One narrow, high-frequency workflow with a clear metric.
- ☐ Success defined up front; ROI measured after launch.

SHIPPING SOMETHING AI-POWERED?

Get a second set of eyes before it hits production.

I help SaaS teams and startups ship reliable, production-grade AI features — evals, grounding, observability and cost, end to end. If any checkbox above is empty, let's talk.

[Book a free fit call →](#)cal.com/yovany/fit-call**Yovany Luis**

AI Product Engineer · ex-Immunefi, Thrive Market · Remote, México

[yovany.dev](#)linkedin.com/in/yovanyluis · [@yovanylg](#)